

Wamu Uganda — Revised Project Plan

Document Type: Strategic & Execution Plan

Version: 3.0

Status: Demo-First Strategy · Jumia Differentiated

Supersedes: Version 2.0

Date: 2026-09-28

1. Executive Summary

Wamu is Uganda's everyday app: one account for local business discovery, chat-based commerce, delivery, and mobility. The original vision was correct, but the execution sequence was wrong twice over.

Mistake 1: Building messaging first would compete directly with WhatsApp without a compelling reason to switch.

Mistake 2: Building a commerce catalog first would look like Jumia — a warehouse, not a neighborhood.

This revised plan corrects both. Wamu will build the **transaction loop first** — but the transaction loop is **conversational and social, not catalog-based**:

Discover → Chat → Buy → Pay → Deliver → Review

Discovery happens through short videos of local businesses. Commerce happens inside chat. Payment happens via mobile money. Delivery happens through a named, verified rider. The same rider network powers passenger mobility. One account carries every role.

The goal is not to clone WhatsApp, TikTok, SafeBoda, or Jumia. The goal is to make them obsolete inside one connected ecosystem.

"Jumia built a warehouse. Wamu is building a neighborhood."

2. Strategic Reframe

2.1 What Wamu Is Not

- Not a general-purpose messenger.
- Not an entertainment-first short-video platform.
- Not a standalone ride-hailing app.
- Not a catalog-based e-commerce platform like Jumia.
- Not a collection of disconnected business tools.

2.2 What Wamu Is

Wamu is Uganda's conversational transaction layer.

It connects local businesses, customers, payments, and verified riders into one seamless experience. Chat, video, and rides are the interfaces. The transaction is the purpose. The neighborhood is the soul.

2.3 The Four Apps Wamu Replaces

App	What It Does Well	How Wamu Wins
WhatsApp	Messaging, groups, calls	Business chat tied to orders, payments, delivery tracking, and rider coordination.
TikTok	Short video, entertainment	Shoppable business videos with a direct "Chat to Order" button — discovery with intent.
SafeBoda	Rides, delivery	One rider network for delivery and rides; higher rider earnings; better customer experience.
Jumia	Catalog commerce	Local, conversational, named merchants; MoMo-first; minutes, not days.

2.4 Why Wamu Is Not Jumia

Jumia

- Product grid
- Anonymous seller
- Cart + checkout
- Warehouse shipping
- 3–5 days delivery

- Cards first
- National
- The product is the hero

Wamu

- Video feed
- Named, verified merchant
- Chat-based ordering
- Neighborhood rider
- Minutes, not days
- MoMo first
- Hyperlocal
- The merchant is the hero

Users will not leave WhatsApp because Wamu has chat. They will leave because Wamu makes business chat *more useful* — it is tied to orders, payments, and delivery. They will not leave TikTok because Wamu has video. They will leave because Wamu's videos are *shoppable*. They will not leave SafeBoda because Wamu has rides. They will leave because Wamu riders earn more. They will not use Wamu over Jumia because it is cheaper. They will use it because it feels like their neighborhood.

3. Product Vision

One Wamu account. Multiple verified capabilities.

- Customer
- Business Owner
- Verified Rider

A person registers once. Additional capabilities are enabled and verified on the same account. No disconnected accounts. No duplicate identity. One Wamu identity, many roles.

3.1 Core Principles

- **Transaction-first:** Every feature must support a real transaction.
- **Conversation-first:** Chat is the store. Cart is replaced by conversation. Checkout is replaced by order cards.
- **Discovery-first:** Short video is the home screen, not a product grid.
- **Uganda-first:** Designed for local payments, connectivity, and mobility.
- **Trust by design:** Verification, security, and privacy built into the architecture.
- **Shared infrastructure:** One rider network, one payment abstraction, one location service.
- **Phased expansion:** Prove one loop, then add reasons to stay.

4. The Core Value Loop

Discover → Chat → Buy → Pay → Deliver → Review

This is the heart of Wamu. Everything else is an extension.

1. **Discover:** Customer scrolls a vertical short-video feed of local businesses.
2. **Chat:** Customer taps "Chat to Order" and opens a conversation with the merchant.
3. **Buy:** Customer negotiates, selects products, and receives an order card in chat.
4. **Pay:** Customer pays via MTN MoMo, Airtel Money, or cash on delivery.
5. **Deliver:** A named, verified rider picks up and delivers the order. Customer tracks live.
6. **Review:** Customer rates the merchant and the rider.

Once this loop works repeatedly, Wamu adds:

- Passenger mobility via the same rider network
- Merchant settlement and rider payouts
- Groups and communities for merchants
- Intelligence and personalization

5. Beachhead Strategy

Start in Kampala. Start with food and groceries.

Why food and groceries?

- High frequency — daily or near-daily use.
- Clear delivery need — already handled by boda riders.
- Existing behavior — customers already chat with food vendors on WhatsApp.
- Easy to measure repeat rate — you know within a week if the loop works.
- Simpler catalog than general retail — fewer SKUs, faster onboarding.
- Visual — food videos perform well in a short-video feed.

Expand to:

1. Pharmacy
2. Retail and electronics
3. General commerce
4. Passenger mobility

Do not launch nationwide. Own one city first.

6. The Demo-First Strategy

The first production artifact is not a launchable product. It is a **fundable demo**.

The demo must prove three things in under 10 minutes:

1. **The loop works** — Discover → Chat → Buy → Pay → Deliver → Review, end to end.
2. **It is Uganda-first** — mobile money, boda riders, local merchants, low data.
3. **It is not Jumia** — video feed, chat ordering, named verified rider, shared rider network.

6.1 What the Demo Must Contain

- Short-video Discover feed of local businesses
- Chat-based ordering with order cards
- Mocked MTN MoMo payment flow
- Named, verified rider with live tracking
- Shared rider network: same rider does delivery and a ride
- Ratings for both merchant and rider

6.2 What the Demo Does Not Contain

- Real payment provider integration
- Real SMS OTP
- Voice and video calls
- Groups and communities
- Merchant settlement and payouts
- ML recommendations
- Multi-city support
- App store submission

Every hour spent on the "does not contain" list is an hour stolen from the demo.

7. Revised MVP Definition

The first production MVP must prove the transaction loop. It must not attempt the entire long-term platform.

7.1 In Scope

Customer App

- Phone OTP login (mocked for demo, real later)
- **Short-video Discover feed** of local businesses
- Like, comment, share, save, follow
- **Chat with merchant** — the primary ordering interface
- **Order cards** sent inside chat
- Pay via MTN MoMo, Airtel Money, or cash on delivery
- Live delivery tracking with named rider
- Rate order, merchant, and rider
- Wamu Move — request a ride using the same account

Merchant App / Web

- Onboarding and verification

- Store profile with verified badge
- **Post short videos** to Discover feed
- Receive, accept, or reject orders via chat
- Mark orders as ready
- Basic sales analytics

Rider App

- Onboarding with ID, selfie, vehicle, and plate verification
- **Availability toggles:** Available for Delivery / Available for Rides
- Accept delivery or ride requests
- Navigate to pickup and dropoff
- Proof of delivery
- Earnings summary

Admin Panel

- Verify merchants and riders
- Manage disputes and refunds
- Monitor fraud
- Basic content moderation

7.2 Out of Scope for MVP

- General 1:1 messaging between users
- Groups and communities
- Status / stories
- Voice and video calls
- Passenger transport beyond one basic ride in the demo
- ML-based recommendations
- Automated merchant settlement and rider payouts
- Multi-city support

These are future phases.

8. Phased Roadmap

Phase	Timeline	Scope	Goal
Phase 0: Concierge	Weeks 0–6	10 merchants, 20 riders, manual dispatch via WhatsApp/phone.	Validate delivery demand. Measure repeat orders. No app.
Phase 1: Demo Build	Weeks 6–16	Flutter demo with video feed, chat ordering, mocked MoMo, named rider tracking, and one ride.	Win funding. Prove the loop and the differentiation.
Phase 2: Production MVP	Months 4–8	Real MoMo/Airtel integration, real OTP, push notifications, hardening.	100 deliveries/day in one Kampala neighborhood.
Phase 3: Neighborhood Density	Months 8–12	Expand to 5 neighborhoods. Merchant settlement. Rider payouts.	500 deliveries/day. 30% repeat rate.
Phase 4: Passenger Mobility	Year 2	Full ride-hailing on the shared rider network. Safety features.	Riders earn more. Customers use one app for everything.
Phase 5: Full Ecosystem	Year 2+	Settlement, ML recommendations, fraud detection, demand prediction.	Everything on Wamu.

Key change from v2.0: The demo is no longer "Phase 1 MVP with video coming later." The demo *is* the video-first, chat-first product. The differentiators are the demo. This is the single most important strategic correction.

9. Functional Specifications

9.1 Account & Capability Model

A person registers once. Additional capabilities are enabled and verified on the same account.

Wamu Account

- └─ User (base)
- └─ Customer (default)
- └─ Business Owner (optional, verified)
- └─ Verified Rider (optional, verified)

9.2 Order Lifecycle

PENDING → CONFIRMED → PROCESSING → READY → OUT_FOR_DELIVERY → DELIVERED

Cancellation:

PENDING → CANCELLED
CONFIRMED → CANCELLED

Refund:

DELIVERED → REFUNDED

Order state transitions must be controlled by the backend. The mobile client must never be trusted to mark an order delivered or a payment successful.

9.3 Delivery States

REQUESTED → RIDER_ASSIGNED → ACCEPTED → GOING_TO_PICKUP → PRODUCT_COLLECTED → IN_TRANSIT

9.4 Rider Verification States

DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED → RESUBMIT → SUSPENDED / EXPIRED

9.5 Rider Operating States

OFFLINE → AVAILABLE → REQUEST_RECEIVED → ACCEPTED → EN_ROUTE_PICKUP → AT_PICKUP → IN_TRANSIT

9.6 Ride Lifecycle (Move)

REQUESTED → RIDER_ASSIGNED → ACCEPTED → EN_ROUTE_PICKUP → AT_PICKUP → IN_TRANSIT → (

10. Technical Architecture

10.1 Stack

- **Mobile:** Flutter (Android-first)
- **Backend:** Python + FastAPI
- **Database:** PostgreSQL
- **ORM:** SQLAlchemy 2.x + Alembic
- **Validation:** Pydantic v2
- **Cache/Queue:** Redis
- **Real-time:** WebSockets
- **Media:** S3-compatible object storage + CDN
- **Push:** Firebase Cloud Messaging
- **Maps:** Google Maps behind Location Service abstraction
- **Payments:** MTN MoMo, Airtel Money, cards behind Payment Provider abstraction
- **Deployment:** Docker, Nginx, CI/CD
- **Monitoring:** Sentry, Prometheus/Grafana
- **No Kubernetes initially**

10.2 Architecture Pattern

Modular monolith, not microservices.

Bounded contexts:

- Identity
- Catalog
- Orders
- Payments

- Delivery
- Mobility
- Discovery (short video)
- Messaging (chat)
- Notifications
- Admin / Audit

10.3 Key Patterns

- Outbox pattern for reliable events
- Idempotency keys for payments and orders
- State machines for order, delivery, ride
- Webhook signature verification
- Feature flags for staged rollout
- Audit logs for sensitive access
- Soft deletes and retention rules

10.4 Initial API Structure

```
POST /api/v1/auth/request-otp
POST /api/v1/auth/verify-otp
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
GET /api/v1/auth/me
GET /api/v1/discover/videos
POST /api/v1/videos/{id}/like
POST /api/v1/videos/{id}/follow

GET /api/v1/conversations
GET /api/v1/conversations/{id}/messages
POST /api/v1/conversations/{id}/messages

POST /api/v1/orders
GET /api/v1/orders/{id}

POST /api/v1/payments
POST /api/v1/payments/webhook

POST /api/v1/deliveries
GET /api/v1/deliveries/{id}
POST /api/v1/deliveries/{id}/accept
POST /api/v1/deliveries/{id}/pickup
POST /api/v1/deliveries/{id}/complete

POST /api/v1/rides
POST /api/v1/rides/{id}/accept
POST /api/v1/rides/{id}/start
POST /api/v1/rides/{id}/complete

POST /api/v1/riders/application
POST /api/v1/riders/documents
GET /api/v1/riders/me
PATCH /api/v1/riders/availability
```

11. Security, Privacy & Compliance

11.1 Security Controls

- Never store raw OTPs.
- Hash refresh tokens.
- Never log OTPs or tokens.
- Private encrypted storage for identity documents.
- Strict authorization for verification data.
- Audit logs for sensitive access.
- Rate limiting by phone, IP, and device.
- File size / type / content validation.
- Server-side authorization.
- Backup and restore testing.
- Fake-account and duplicate-identity controls.
- GPS manipulation detection.
- Fake-trip and payment-fraud controls.
- Delivery dispute workflows.
- Fake-review monitoring.
- Suspicious-device-change monitoring.

11.2 Privacy

- Detailed location history should not be retained forever.
- Active-trip location should be available only for the relevant operational and safety period.
- Government ID images and sensitive verification records must never be publicly displayed.
- Customers see a safe verified profile: name, photo, verified badge, vehicle type, plate, rating, trip count, Wamu rider reference.

11.3 Compliance

- Uganda Data Protection and Privacy Act 2019
- Consent, data subject rights, retention
- KYC / AML for payments
- Transport licensing for passenger mobility
- Rider insurance and liability
- Content moderation for Discover

- Payment provider commercial agreements

12. Payments

12.1 Payment Flow

Customer → Create Order → Create Payment → Provider
Customer Confirmation → Provider Webhook → Wamu Verification
→ Payment SUCCESS → Order CONFIRMED

12.2 Requirements

- Payment-provider abstraction
- Server-side payment confirmation
- Webhook signature verification
- Idempotency
- Duplicate webhook protection
- Refund handling
- Transaction reconciliation

12.3 Supported Providers

- MTN Mobile Money
- Airtel Money
- Cards (subject to API/commercial availability)
- Cash on Delivery

13. Delivery & Rider Network

13.1 Delivery Flow

Customer → Business → Delivery Request → Rider → Customer

Steps:

- Customer delivery address
- Business pickup location
- Delivery request
- Distance and ETA
- Delivery price estimate
- Rider matching
- Rider acceptance
- Pickup confirmation
- Live tracking
- Delivery completion
- Proof of delivery
- Ratings

13.2 Shared Rider Network

Wamu uses one common rider network instead of separate delivery and passenger systems. A verified rider may enable one or more approved service types.

- Boda-boda and other approved vehicle operators can apply.
- Riders can provide delivery.
- Riders can provide passenger transport.
- Service types can be enabled or disabled at any time.
- Only eligible and available riders receive matching requests.

13.3 Rider Matching

Initial matching should be simple and explainable:

- Verified status
- Availability
- Service capability
- Distance to pickup
- ETA

- Vehicle type
- Current trip status
- Service area

Then become more data-driven later.

13.4 Pricing

Backend Pricing Engine:

```
Delivery = base + distance + time + service factors
Passenger = base + distance + time + vehicle factors
```

Flutter requests an estimate but does not define the final fare. The system distinguishes estimated fares from final fares unless a specific pricing policy guarantees the amount.

14. Discover — Short Video

Discover is a vertical short-video experience focused on Ugandan businesses, products, and services. Businesses publish videos showing products, services, promotions, locations, and everyday activity.

- Autoplay vertical feed
- Business profile and product/service links
- **"Chat to Order" button** — the primary CTA (not "Buy")
- Likes, comments, shares, saves, follows
- Search and category discovery
- Local / nearby discovery
- "Not Interested" feedback
- Content reporting and moderation

14.1 Recommendation Engine

The first version should use rule-based ranking. Machine learning should be introduced after enough interaction data exists.

Ranking signals:

- Followed businesses
- Previously engaged businesses
- Related categories
- Similar businesses
- Local businesses
- New and recently active businesses
- Trending content
- Exploration candidates

Important signals include watch duration, completion, replay, likes, comments, shares, saves, follows, business visits, product clicks, searches, orders, and negative feedback.

Illustrative starting score:

```
score = interest * 0.30
        + watch * 0.20
        + engagement * 0.15
        + freshness * 0.10
        + local_relevance * 0.10
        + business_quality * 0.05
        + exploration * 0.10
        - repetition_penalty
        - negative_feedback
```

The weights are starting parameters, not permanent product decisions.

15. Transactional Chat

Chat is the ordering interface. It is not a general-purpose messenger.

- One-to-one chat between customer and merchant
- Order cards sent inside chat
- Payment buttons inline
- Delivery tracking inline
- Receipts and delivery updates inside chat
- Merchant groups (Phase 4)
- Business chat tied to orders, payments, and delivery

Goal: Replace WhatsApp for business communication because Wamu chat is more useful — it is connected to orders, payments, and delivery.

16. Passenger Mobility — Wamu Move

Wamu Move uses the same verified rider network as delivery.

Pickup → Destination → Fare Estimate → Ride Request → Rider Match
→ Navigation → Trip → Completion

- Pickup and destination
- Distance and ETA
- Estimated fare
- Rider matching and acceptance
- Live tracking
- Navigation
- Trip history
- Payment
- Passenger / rider ratings

16.1 Safety

- Verified rider and vehicle information
- Active-trip tracking
- Share Trip
- Call / chat rider
- Report Problem
- Emergency Help entry point
- Privacy-controlled trip records

17. Maps and Location

Google Maps can provide mapping, geocoding, routing, distance, ETA, and navigation capabilities. Wamu should place these behind its own Location Service abstraction.

Wamu Location Service

- Geocoding
- Reverse geocoding
- Route calculation
- Distance / ETA
- Business locations
- Navigation integration

This shared service supports Business, Delivery, and Move.

18. Metrics

North Star: Completed deliveries per day.

Secondary North Star: Conversations that convert to paid orders.

Supporting metrics:

- Order completion rate
- Average delivery ETA
- Repeat customer rate
- Merchant retention
- Rider utilization
- Payment success rate
- CAC per customer
- Take rate per order
- Safety incidents per 1,000 trips
- Daily / monthly active users
- Messages sent and delivered
- Video uploads and qualified discovery
- Average watch time and completion

- Product clicks from Discover
- Completed rides
- Rider earnings per day

19. Risks & Mitigation

Risk	Impact	Mitigation
Too many features at once	High	Demo-first strategy. Release in phases around the core loop.
Looking like Jumia	High	Video-first, chat-first, named rider. Never lead with a catalog grid.
Poor connectivity	High	Low-data mode, compression, resumable uploads.
Fake identities / fraud	High	OTP, verification, device controls, risk rules.
Sensitive data exposure	High	Private storage, encryption, authorization, audit logs.
Weak recommendations	Medium	Start rule-based; add ML after collecting data.
Payment disputes	High	Server confirmation, webhooks, idempotency, reconciliation.
Rider safety	High	Verification, tracking, reporting, safety controls.
Operational complexity	High	Modular architecture, simple initial infrastructure.
Regulatory issues	High	Legal review for payments, transport, data protection.
No funding	High	Concierge MVP + working demo + targeted grants.

20. Operational Plan

20.1 Concierge Phase (Already Validated)

- 10 merchants
- 20 riders
- Manual dispatch via WhatsApp / phone
- Prove delivery works

- Measure repeat orders
- No app yet

20.2 Demo Build Phase

- 6–10 weeks part-time, 4–6 weeks full-time
- Flutter + FastAPI + PostgreSQL
- Video feed, chat ordering, mocked MoMo, named rider tracking, one ride
- Seed data with 5 users, 3 businesses, 9 products, 6 videos, 2 riders

20.3 Onboarding (Post-Funding)

- Merchant onboarding and photography
- Rider onboarding and training
- Customer support
- Dispute resolution
- Refunds and reconciliation
- Fraud monitoring
- Safety incident response
- Rider earnings and payouts

21. Go-to-Market

21.1 City Launch

- Kampala first
- Food and groceries first
- Then pharmacy, retail, general commerce

21.2 Merchant Acquisition

- Direct sales
- Referral incentives
- Free onboarding and photography

- Competitive take rate

21.3 Rider Recruitment

- Referral bonuses
- Higher earnings through delivery + rides
- Fast verification
- Weekly payouts

21.4 Customer Acquisition

- Referral codes
- First-order discounts
- Social media
- Local influencers
- Business partnerships

22. Conclusion

Wamu's ideal is correct: one app for discovery, chat, commerce, delivery, and mobility. But the execution must be **transaction-first, conversation-first, and demo-first**.

Build the loop:

Discover → **Chat** → **Buy** → **Pay** → **Deliver** → **Review**

Then add rides.

Then add everything.

That is how Wamu gets users off WhatsApp, TikTok, SafeBoda, and Jumia. Not by cloning them. By making them obsolete inside one connected ecosystem.

"Jumia built a warehouse. Wamu is building a neighborhood."

Build the loop. Ship the demo. Win the room. Scale the ecosystem.

Appendix A — Database Tables (MVP)

Core: users, user_profiles, devices, sessions, otp_challenges, businesses, business_profiles, categories, products, services, orders, order_items, payments, payment_events.

Messaging (demo-critical): conversations, conversation_members, messages, message_attachments, order_cards.

Discover (demo-critical): business_videos, video_assets, video_categories, video_hashtags, video_views, video_watch_events, video_likes, video_comments, video_shares, video_saves, business_followers, user_feedback, video_reports.

Delivery & Rider: rider_profiles, rider_verifications, rider_documents, rider_vehicles, rider_vehicle_documents, rider_service_types, rider_service_areas, rider_availability, rider_status_history, delivery_requests, delivery_events.

Mobility: ride_requests, ride_trips, trip_locations, rider_ratings.

Recommendations (Phase 3+): user_interest_profiles, recommendation_candidates, recommendation_impressions.

Appendix B — Environment & Deployment

- Docker
- Nginx
- CI/CD pipeline
- Staging and production environments
- PostgreSQL managed service
- Redis managed service
- S3-compatible object storage
- CDN
- Sentry
- Prometheus / Grafana
- Firebase Cloud Messaging
- Maps API
- Payment provider APIs

No Kubernetes in first deployment unless real operational scale requires it.

Appendix C — What Changed From v2.0

Area	v2.0	v3.0
MVP shape	Catalog + browse + order + deliver	Video feed + chat ordering + order cards + shared rider
Discovery	Merchant list / categories	Short-video feed (TikTok-style, shoppable)
Ordering	Product grid, cart, checkout	Chat, negotiation, order cards
Video	Phase 3 (future)	Home screen of the demo
Chat	Phase 2 (future)	Primary ordering interface
Rides	Year 2	One ride included in the demo
Positioning	Transaction layer	Neighborhood vs. warehouse
Jumia	Not addressed	Explicitly differentiated
North Star	Completed deliveries per day	Completed deliveries + conversations that convert
Demo	Not specified	Fully specified, screen by screen

End of Document — Wamu Revised Project Plan v3.0